CS 355 - Topic 3: Asynchronous Programming

Dates: February 18, 2026 and February 23, 2026

Why do we need asynchronous programming?

- We are accustomed to a procedural style of programming where statements are executed top down. Code written in this style share the feature that they are blocking
- **Blocking**: The next task cannot be completed until the previous task has completed
- This style of programming may work fine for local applications thar run near instantaneously, but when we have network calls in the mix, this could cause our application to get stuck stalling as it waits for data

```
taskA(); // Network request
taskB();
```

- In the above code block, imagine the network request in `taskA` takes an extended amount of time to complete. `taskB` will be waiting for `taskA` to complete to start. This is extremely inefficient as a network instabilitiy could cause our entire application to be locked up
- Example of Procedural Programming:

```
for(let i = 0; i < 1000000000; i++){
    // Intentionally blank
}
console.log("Finished");
```

- While the above code block is executing you would NOT be able to interact with it's corresponding webpage (absolutely all features will disabled). After 10 or so second when the for loop finishes and and the "Finished" statement prints, all blocked commands the user tried to perform during the disabled period quickly fire. This is a horrible user experience and a horrible way for an application to function
- Example of Asynchronous Programming:

```
setTimeout(
    function(){
        console.log("Finished");
    }
, 10000);
```

- The above code block is far superior to the one directly above it due to the fact the user is able to interact with the page and everything works as intended and after roughly 10 seconds or so "Finished" is printed

What are callbacks?

- A **callback** is a function passed as an argument to another function to be executed later (once some task completes)
- In the last code block (the one featuring `setTimeout`), the anonymous function passed in is a callback and it gets called back after the timer expires
- Callbacks are the foundation of asynchronous programming in JavaScript

What is the event loop?

- JavaScript by default is single-threaded meaning it can only do one thing at a time
- JavaScript achieves asynchronous behavior through the event loop, which coordinates between the call stack and a task queue
- When an asynchronous operation is triggered (such as `setTimeout` or any other network request), it is offloaded to an external runtime environment (this external runtime would be the browser if your code runs on a webpage, and it would Node.js if the code runs on a server) to handle in the background while the JavaScript code continues executing
- Once the asynchronous operation completes, its callback is placed in the event queue
- The event loop continuously checks to see if the call stack is empty, if it is ever empty, it picks up the next callback from the queue and executes it
- This is why in the earlier example `setTimeout` did not freeze the page because the timer ran in the background and the rest of the code kept executing normally
- Flow diagram of the event loop:

```
Call Stack → runs synchronous code
     ↓
Async task offloaded to runtime (e.g. timer, network call)
     ↓
Callback placed in event queue when done
     ↓
Event Loop moves callback to Call Stack when stack is empty
```

What is Node.js?

- The Node.js runttime is powdered by the open source V8 JavaScript engine
- JavaScript is single-threaded
- As a developer, you can make specific calls to Node.js's API's, which uses threads to accomplish asynchronous programming
- API (Application Programming Interface): A collection of public functions, methods, and variables that form a way to interact with some (typically built in) utility
- Typically we can use API's by just calling their public interface methods like `fs.readFile()` or `navigator.geolocation.getCurrentPosition()`

Node.js Architecture

- Layer 1: Node.js Application
- Layer 2: Node.js API (JavaScript)

- Layer 3: Node.js Bindings (JavaScript to C/C++) | Node.js Standard Library (Core Modules) | C/C++ AddOns
- Layer 4: V8 (JavaScript Engine) | LibUv (Library) | c-ares | llhttp/http-parser | open-ssl | zlib
- Layer 5: Operating System

Node.js API

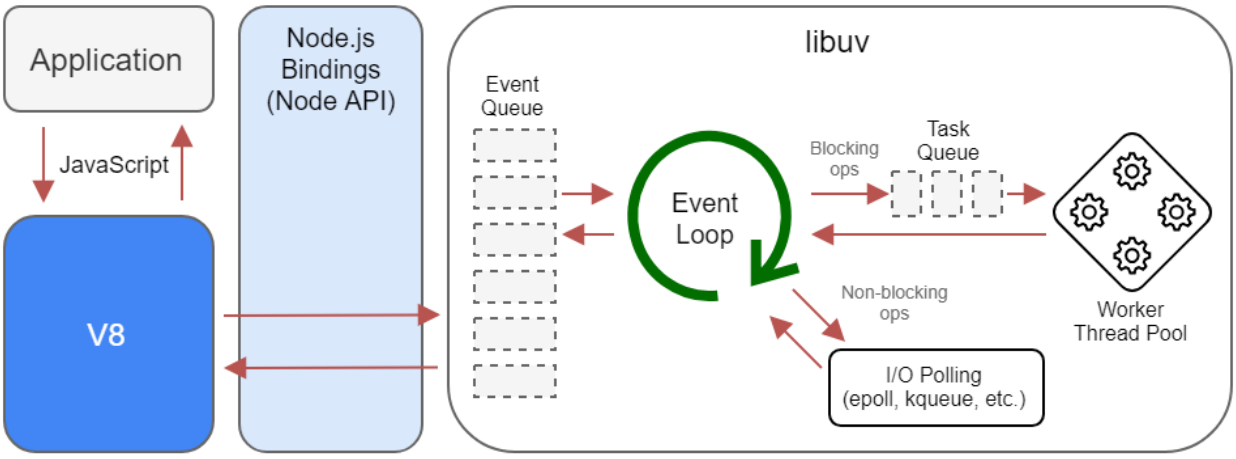
- Node.js's API's provides public interfaces that allow our application code to communicate qith the operating system in a limited manner
- This abstraction model allows the application programmer to utilize system resources without concerning themselves with how those resources got obtained
- Node.js Documentation: this link contains all the built-in functions we can utilize with this API

V8 (JavaScript Engine)

- A JavaScript engine is a program that executes JavaScript code
- Modern JavaScript engines contain both an interpreter and a compiler

Libuv

- Libuv is a C library that provides support for asynchronous I/O based operations using the event loop
- When our application makes an asynchronous call, that task (whether it be reading a file or making a network call) is given to Liuv and execution of the program continutes without waiting for a response
- Libuv will either take this task and push it onto the `task queue` if it requires the CPU to complete, otherwise if the task is IO related it will handle that task seperately
- The tasks in the `task queue` will wait for a `worker thread` to open up. These worker threads will pick up tasks as they become free
- On the otherhand, IO bound tasks will be checked for updates at set intervals by the event loop
- There are two parts to asynchronous calls:
  - The task that you want to complete
  - What will occur when that task is finished (aka the callback)
- The callback is placed on the event queue
- What the event loop does is it checks for when the program's (V8's) call stack is empty, when it's empty it takes the element at the front of the event queue and adds it to the call stack
- Libuv Diagram:



Example of Callback

```
const fs = require("fs");
fs.readFile("input/file01.txt", "utf8", do_after_reading);
function do_after_reading(err, data){
    if(err){
        console.log(`Error Reading File`);
    }
    else{
        console.log(`Finished Reading File: `, data.length);
    }
}
```

- The code reads a file called file01.txt as if it were utf8 text and once it's finished reading we are going to run a function called `do_after_reading`
- In this case, reading a file is our asynchronous task. Once that task is complete the callback function `do_after_reading` is added to the event queue. It stays on the event queue until the event loop identifies that V8's call stack is empty. When the call stack is completely empty, the event loop moves the function from the event queue to the call stack and then it is executed. In the case our inpput file simply had `"Hello World"`, our console would print `Finished Reading File: 11`

DNS Resolution

- Domain Name System (DNS) is an internet service that converts human readable domains into machine readable IP addresses
- We catch fetch the IP address associated to a domain. For example, `google.com` is the equivalent to `172.217.19.164`. How much time it takes us to fetch this information depends on our internet speed, whether the connection is wired or wireless, how many servers the request needs to travel through, and if there is record cached at intermediary already. It is also possible for the request to never complete thus why Node.js has timeout option for this
- Our first DNS request:

```
const dns = require("dns");
const domain = "venus.cs.qc.cuny.edu";
dns.resolve(domain, after_resolution);
function after_resolution(err, records){
    if(err){
        console.error("Failed to resolve", domain);
    }
    else {
        console.log(domain, records);
    }
}
```

- The callback function accepts two arguments: `err` and `records`. These arguments are the return values from the previously completed asynchronous code. The argument names don't matter but their order does. Node.js typically returns errors as the firsr argument. Errors are an object type so

you can look at the error details in the `err` object. The secod parameter typically represents the data that was returned from the request
- Also, the function declaration for the callback function does not need to be after all the code, even if it was at the start it works the exact same
- The output of the above program will be `venus.cs.qc.cuny.edu [ '149.4.99.190' ]`
    - The data is in braces because a domain can have multiple IP addresses

Multipe Synchronous Tasks

- The following code block would be an error because a synchronous task is trying to use a variable defined inside of a asynchronous task. This will be an error because synchronous tasks ALWAYS complete before an asynchronous task even begins

```
const dns = require("dns");
const domain = "venus.cs.qc.cuny.edu";
let data;

dns.resolve(domain, after_resolution);
function after_resolution(err, records){
    if(err){
        console.error("Failed to resolve", domain);
    }
    else{
        data = records;
    }
}

next_task(data);
function next_task(input){
    console.log(input); // data is passed in as input and data in
undefined
}
```

- We can fix the above code block by implementing continuation passing style. Corrected code:

```
const dns = require("dns");
const domain = "venus.cs.qc.cuny.edu";
let data;

dns.resolve(domain, after_resolution);
function after_resolution(err, records){
    if(err){
        console.error("Failed to resolve", domain);
    }
    else{
        data = records;
        next_task(data);
    }
}
```

```
function next_task(input){
    console.log(input);
}
```

Example of Continuation Passing Style

```
const dns = require("dns");
const venus = "venus.cs.qc.cuny.edu";
const mars = "mars.cs.qc.cuny.edu";
const ip_addresses = [];

dns.resolve(venus, after_venus);
function after_venus(err, records){
    if(err){
        console.error("Failed to resolve", venus);
    }
    else{
        ip_addresses.push(records);
    }
    dns.resolve(mars, after_mars);
}
function after_mars(err, records){
    if(err){
        console.error("Failed to resolve", mars);
    }
    else{
        ip_addresses.push(records);
    }
    console.log(ip_addresses.join(", "));
}
```

- The order in this scenario is guaranteed, if both exist, the IP address of `venus` will ALWAYS print before `mars`

Concurrency

```
const dns = require("dns");
const venus = "venus.cs.qc.cuny.edu";
const mars  = "mars.cs.qc.cuny.edu";

dns.resolve(venus, after_venus);
console.log("Prints Immediately 01");
function after_venus (err, records){
    if(err){
        console.error("Failed to resolve", venus);
    }
    else{
        console.log(venus, records);
```

```
    }
}

dns.resolve(mars, after_mars);
console.log("Prints Immediately 02");
function after_mars(err, records){
    if(err){
        console.error("Failed to resolve", mars);
    }
    else{
        console.log(mars, records);
    }
}
```

- The order the two `console.log("Prints Immediately")` is guranteed, HOWEVER the order of the print statements from dns.resolve() is NOT guranteed

Closures

- In the previous code block you notice that both functions `after_venus` and `after_mars` have roughly the same code. We can generalize them into a single function `after_resolution(err, records)`. We achieve this by implementing functions closures like so:

```
const dns = require("dns");
const venus = "venus.cs.qc.cuny.edu";
const mars = "mars.cs.qc.cuny.edu";

function resolve(domain){
    dns.resolve(domain, after_resolution);
    function after_resolution(err, records){
        if(err){
            console.error("Failed to resolve", domain);
        }
        else{
            console.log(domain, records);
        }
    }
};

resolve(venus);
resolve(mars);
```